

Отчет по лабораторной работе №7

Основы информационной безопасности

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Ответы на контрольные вопросы	10
5	Выводы	12

Список иллюстраций

3.1	Функция генерации ключа	7
3.2	Функция для шифрования текста	7
3.3	Подбор возможных ключей для фрагмента	8

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования

2 Задание

Нужно подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!». Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Приложение должно: 1. Определить вид шифротекста при известном ключе и известном открытом тексте. 2. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста

3 Выполнение лабораторной работы

Я выполняла лабораторную работу на языке программирования Python, листинг программы и результаты выполнения приведены в отчете.

Требуется разработать программу, позволяющее шифровать и дешифровать данные в режиме одноразового гаммирования. Начнем с создания функции для генерации случайного ключа

```
def en_de_crypt(text, key):  
    new_text = ''  
    for i in range(len(text)):  
        new_text += chr(ord(text[i]) ^ ord(key[i % len(key)]))  
    return new_text
```

Рис. 3.1: Функция генерации ключа

Необходимо определить вид шифротекста при известном ключе и известном открытом тексте. Так как операция исключающего или отменяет сама себя, делаем одну функцию и для шифрования и для дешифрования текста

```
def find_possible_key(text, fragment):  
    possible_keys = []  
    for j in range(len(text) - len(fragment) + 1):  
        possible_key = ''  
        for i in range(len(fragment)):  
            possible_key += chr(ord(text[j+i]) ^ ord(fragment[i]))  
        possible_keys.append(possible_key)  
    return possible_keys
```

Рис. 3.2: Функция для шифрования текста

Нужно определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных

вариантов прочтения открытого текста. Для этого создаю функцию для нахождения возможных ключей для фрагмента текста

```
t = 'С Новым Годом, друзья!'
key = generate_key_hex(t)
en_t = en_de_crypt(t, key)
de_t = en_de_crypt(en_t, key)
keys_t_f = find_possible_key(en_t, 'С Новым')
fragment = 'С Новым'
print('Открытый текст: ', t, "\nКлюч: ", key, "\nШифротекст: ", en_t, "\nИсходный текст: ", de_t, )

print('Возможные ключи: ', keys_t_f)
print('Расшифрованный фрагмент: ', en_de_crypt(en_t, keys_t_f[0]))

Открытый текст: С Новым Годом, друзья!
Ключ: ApV5aBX756a1R3P1xoVYII
Шифротекст: QPyfMBBl7uJ3fрйыЕIh
Исходный текст: С Новым Годом, друзья!
Возможные ключи: ['ApV5aBX', 'fa\x16e;/M', 'jм7Vd\x1a', 'a\x14Zknd', 'r@Щ\x14Cі', '{@0\x18:\x1e3', 'E7;6gDR', 'xI\x15k=6u', '\x07BH1\3u', 'v\x12Pen
a', 'tRsjr\x16\x04', 'юююю\x10', 'Ofac\ng]', 'чP@\x00\x1e*', 'e0%\x125^', "[ИI_!e"]
Расшифрованный фрагмент: С НовымВи92b>130Ei9)
```

Рис. 3.3: Подбор возможных ключей для фрагмента

Листинг программы 1:

```
import random
import string

def generate_key_hex(text):
    key = ''
    for i in range(len(text)):
        key += random.choice(string.ascii_letters + string.digits) #генерация цифры дл
    return key

#для шифрования и дешифрования
def en_de_crypt(text, key):
    new_text = ''
    for i in range(len(text)): #проход по каждому символу в тексте
        new_text += chr(ord(text[i]) ^ ord(key[i % len(key)]))
    return new_text

def find_possible_key(text, fragment):
    possible_keys = []
```



```

    for i in range(len(text) - len(fragment) + 1):
        possible_key = ""
        for j in range(len(fragment)):
            possible_key += chr(ord(text[i + j]) ^ ord(fragment[j]))
        possible_keys.append(possible_key)
    return possible_keys

t = 'С Новым Годом, друзья!'
key = generate_key_hex(t)
en_t = en_de_crypt(t, key)
de_t = en_de_crypt(en_t, key)
keys_t_f = find_possible_key(en_t, 'С Новым')
fragment = "С Новым"

print('Открытый текст: ', t, "\nКлюч: ", key, "\nШифротекст: ', en_t, '\nИсходный текст: ', de_t)

print('Возможные ключи: ', keys_t_f)
print('Расшифрованный фрагмент: ', en_de_crypt(en_t, keys_t_f[0]))

```

4 Ответы на контрольные вопросы

1. Поясните смысл однократного гаммирования. - Однократное гаммирование - это метод шифрования, при котором каждый символ открытого текста гаммируется с соответствующим символом ключа только один раз.
2. Перечислите недостатки однократного гаммирования. - Недостатки однократного гаммирования:
 - Уязвимость к частотному анализу из-за сохранения частоты символов открытого текста в шифротексте.
 - Необходимость использования одноразового ключа, который должен быть длиннее самого открытого текста.
 - Нет возможности использовать один ключ для шифрования разных сообщений.
3. Перечислите преимущества однократного гаммирования. - Преимущества однократного гаммирования:
 - Высокая стойкость при правильном использовании случайного ключа.
 - Простота реализации алгоритма.
 - Возможность использования случайного ключа.
4. Почему длина открытого текста должна совпадать с длиной ключа? - Длина открытого текста должна совпадать с длиной ключа, чтобы каждый символ открытого текста гаммировался с соответствующим символом ключа.

5. Какая операция используется в режиме однократного гаммирования, назовите её особенности? - В режиме однократного гаммирования используется операция (исключающее ИЛИ), которая объединяет двоичные значения символов открытого текста и ключа для получения шифротекста. Особенность XOR - если один из битов равен 1, то результат будет 1, иначе 0.
6. Как по открытому тексту и ключу получить шифротекст? - Для получения шифротекста по открытому тексту и ключу каждый символ открытого текста гаммируется с соответствующим символом ключа с помощью операции XOR.
7. Как по открытому тексту и шифротексту получить ключ? - По открытому тексту и шифротексту невозможно восстановить действительный ключ, так как для этого нужна информация о каждом символе ключа.
8. В чем заключаются необходимые и достаточные условия абсолютной стойкости шифра - Необходимые и достаточные условия абсолютной стойкости шифра:
- Ключи должны быть случайными и использоваться только один раз.
 - Длина ключа должна быть не менее длины самого открытого текста.
 - Ключи должны быть храниться и передаваться безопасным способом.

5 Выводы

В ходе выполнения данной лабораторной работы мной было освоено на практике применение режима однократного гаммирования.